Engineering, Built Environment and IT
Department of Computer Science
Programming Languages
COS 333

Semester Test 2
14 May 2024

Examiner: Mr W. S. van Heerden

Instructions:

Read the questions carefully and answer all questions.
This section comprises of **20** questions, and a total of **35** marks.
You have **1.5** hours to complete this test.
This test is an *online* assessment:
The test will automatically submit when the time (1.5 hours) expires.
You can also submit the test yourself if you are done ahead of time.
This test is **closed book** and is subject to the University of Pretoria integrity statement provided below.
You are not allowed to consult any sources other than the Practical 3 and 4 specifications and the SWI-Prolog Reference Manual (provided as documentation for Practical 3).
For practical implementation questions, your Prolog submission must be working Prolog code that will successfully interpret using version 9.0.4 of the SWI-Prolog interpreter. You may (and should) use this interpreter to test your submissions. Once you have finished writing each program, make sure each is saved in a text file with the appropriate name (described in the question), and submit it to the correct upload slot.
You may use a non-programmable calculator.
You may not use any other programmable devices.

Integrity Statement:

The University of Pretoria commits itself to producing academic work of integrity. I affirm that I am aware of and have read the Rules and Policies of the University, more specifically the Disciplinary Procedure and the Tests and Examinations Rules, which prohibit any unethical, dishonest or improper conduct during tests, assignments, examinations and/or any other forms of assessment. I am aware that no student or any other person may assist or attempt to assist another student, or obtain help, or attempt to obtain help from another student or any other person during tests, assessments, assignments, examinations and/or any other forms of assessment.

Timed Test	This test has a time limit of 1 hour and 30 minutes.This test will save and be submitted automatically when the time expires. Warnings appear when **half the time**, **5 minutes**, **1 minute**, and **30 seconds** remain.*[The timer does not appear when previewing this test]*
Multiple Attempts	Not allowed. This Test can only be taken once.
Force Completion	This Test can be saved and resumed at any point until time has expired. The timer will continue to run if you leave the test.

QUESTION 1

1. Assume that the following Prolog-like program code is provided:

```
tennis(angela).
soccer(gary).
sportsPerson(X) :- soccer(X).
sportsPerson(X) :- tennis(X).
```

Also assume that the following query is provided:

```
?- sportsPerson(angela).
```

Select from the following options to **complete** the following steps that are performed from the first step to the last one, if it is assumed that bottom-up resolution (or forward chaining) is performed to prove the query. Note that standard Prolog implementations do not use bottom-up resolution (they use top-down resolution). Select "No step performed" for any steps you do not use.

Step 1: [ ]

Step 2: [ ]

2 points

QUESTION 2

1. Assume that you have a Prolog program containing three kinds of facts related to all the animals in a city. The first type of fact has the form `cat(X)` and means that the atom `X` is a cat. The second type of fact has the form `dog(X)` and means that the atom `X` is a dog. The third type of fact has the form `pet(X, Y)` and means that the atom `X` is a pet owned by a person represented by the atom `Y`. You wish to write a query that determines whether `peter` owns a pet that is a cat. Consider the following queries, and **select** the one that most efficiently determines an instantiation for the pet of `peter`.

- `?- cat(X), pet(X, peter).`
- `?- pet(X, peter), cat(X).`
- ```
  ?- cat(X).
  ?- pet(X, peter).
  ```
- ```
  ?- pet(X, peter).
  ?- cat(X).
  ```

1 points

QUESTION 3

1. Assume that you have Prolog facts describing tenants and addresses, taking the following form:

```
tenant(alice, tony).
tenant(tom, jane).
tenant(joe, tony).
tenant(mary, jane).
address(alice, pineStreet12).
address(tom, shillingLane15).
address(joe, duncanRoad6).
address(mary, shillingLane15).
address(jonathan, pineStreet12).
```

Here, `tenant(X, Y)` means that person `X` is the tenant of property owner `Y` (meaning that `X` rents a flat from `Y`), and `address(X, Y)` means that person `X` currently lives at address `Y`.

Write a Prolog proposition called `ownsSharedProperty(X)`, which succeeds if atom `X` is the owner of a property rented by two individuals, meaning that `X` has two different tennants with the same address. In the above example, only `jane` owns a shared property. This is because `tom` and `mary` are both tenants of `jane`, and both `tom` and `mary` have `shillingLane15` as an address. Note that, even though `alice` and `jonathan` both have `pineStreet12` as an address, this does not mean that `tony` owns a shared property. This is because `alice` is a tenant of `tony`, but `jonathan` is not (implying that `jonathan` is just visiting `alice`). Therefore, no other atoms satisfy the `ownsSharedProperty` proposition in this example. You may assume that no person is a tenant of more than one property owner, and that no person lives at multiple addresses.

Hint 1: When testing your proposition, use variables, such as in the following query:

```
?- ownsSharedProperty(X)
```

Then use Prolog's support for re-querying to determine all the atoms that satisfy the `ownsSharedProperty` proposition.

Hint 2: The built-in `not` predicate can be applied to propositions. For example, if the proposition `parent(peter, tina)` is true, then `not(parent(peter, tina))` will be false.

Submit your program code in a file named `s99999999.pl` (where `99999999` is your student number) to the assignment submission slot labelled "Semester Test 2: Simple Prolog Proposition", and select the "Yes" answer below once you have done so. If you do not make a submission, select the "No" answer below.

Take careful note of the following requirements:

- You may not define any additional facts other than `tenant` and `address` facts. If you do so, you will forfeit all marks for this question. You are required to write one or more rules that together define the cousin proposition.
- Your submission must be working Prolog code that will be successfully interpreted by the SWI-Prolog interpreter installed on the Windows computers in the Informatorium. You may (and should) use this SWI-Prolog interpreter to test your submission. For notes on using the interpreter, and re-querying, see the specification for Practical 3. Once you have finished writing your program, save it in a text file with the appropriate name, and submit it to the correct upload slot.
- You may define additional helper propositions.
- **Note that you may only use the simple constants, variables, list manipulation methods, arithmetic and relational expressions, and built-in predicates discussed in the textbook and slides**. In particular, do not use if-then, if-then-else, and similar constructs. You may NOT use any more complex predicates provided by the Prolog system itself. In other words, you must write all your own propositions. **Failure to observe this rule will result in all marks for this question being forfeited**.

○ Yes

○ No

5 points

QUESTION 4

1. **Write** a Prolog proposition called `countNonMatching(E, L, C)`, which succeeds if `C` is the number of elements in the list `L` that do not match the atom `E`. You may assume that list `L` is a simple list containing only atoms. For example, the query:
   `?- countNonMatching(a, [a, a], X).`
   should be true with the instantiation `X = 0`. This is because there are no elements that do not match the atom `a` in the list `[a, a]`. Similarly, the query
   `?- countNonMatching(a, [a, b, a, c, d], X).`
   should be true with the instantiation `X = 3`. This is because `b`, `c`, and `d` do not match the atom `a` in the list `[a, b, a, c, d]`.
   Submit your program code in a file named `s99999999.pl` (where `99999999` is your student number) to the assignment submission slot labelled "Semester Test 2: List Processing Prolog Proposition", and select the "Yes" answer below once you have done so. If you do not make a submission, select the "No" answer below.

   Take careful note of the following requirements:

- Your submission must be working Prolog code that will be successfully interpreted by the SWI-Prolog interpreter installed on the Windows computers in the Informatorium. You may (and should) use this SWI-Prolog interpreter to test your submission. For notes on using the interpreter, and re-querying, see the specification for Practical 3. Once you have finished writing your program, save it in a text file with the appropriate name, and submit it to the correct upload slot.
- You may define additional helper propositions.
- **Note that you may only use the simple constants, variables, list manipulation methods, arithmetic and relational expressions, and built-in predicates discussed in the textbook and slides**. In particular, do not use if-then, if-then-else, and similar constructs. You may NOT use any more complex predicates provided by the Prolog system itself. In other words, you must write all your own propositions. **Failure to observe this rule will result in all marks for this question being forfeited**.

- ○ Yes
- ○ No

5 points

QUESTION 5

1. Consider the following programming languages, and **select** the one that is the most writable in terms of name length, in a real-world context.
   - ○ Fortran 95
   - ○ C99
   - ○ C++
   - ○ Ada

1 points

QUESTION 6

1. Consider the following program code in a hypothetical programming language using syntax similar to C++:

```
1.  class MyClass {
2.     int count;

3.     MyClass(int aCount) {
4.        count = aCount
5.     }

6.     int getCount() {
7.        return count
8.     }
9.  }

10. void main() {
11.    int class = 5
12.    MyClass m(class)
13. }
```

In the above code, line numbers are provided for reference purposes. **Identify** the type of special words used in this programming language if the following conditions are met:

- A compilation error occurs on line 11: _____
- No compilation error occurs: _____

2 points

QUESTION 7

1. Consider the following program code in a hypothetical programming language:

```
1. myVariable = [1, 2, 3]
2. myVariable = 14
```

Line numbers are provided for reference purposes. Assume that the above program code results in a compilation error on line 2. Consider the following options, and **select** only the kind of type binding that is most likely be used for `myVariable`.

- ○ Explicit type declaration.
- ○ Implicit type declaration without type inferencing.
- ○ Implicit type declaration with type inferencing.
- ○ Dynamic type binding.

1 points

QUESTION 8

1. Consider the following categories of variables, and **select** the one that has no name associated with it.
   - ○ Static variables.
   - ○ Stack-dynamic variables.
   - ○ Explicit heap-dynamic variables.
   - ○ Implicit heap-dynamic variables.

1 points

QUESTION 9

1. Consider the following program code in a hypothetical programming language (line numbers are provided for reference purposes):

```
1. void doSomething()
2.     myValue = 2
3.     print myValue
4.     myValue = [5, 7, 9]
5.     print myValue

7. int main()
8.     doSomething()
9.     return 0
```

Assume that the programming language uses static scope, that variables are not explicitly declared, that the `main` subprogram is the first thing to be executed when the program runs, and that indentation indicates the body of subprograms. **Fill in** the start and end of the scope of the variable `myValue`, and the lifetime of the single integer value stored by the variable `myValue`, using the provided line numbers.

Scope of `myValue`: From _____ up to and including _____

Lifetime of the single integer value stored by `myValue`: From _____ up to and including _____

4 points

QUESTION 10

1. Consider the following program code in a hypothetical programming language:

```
void f1() {
   var X = 8
   call f2()
}

void f2() {
   print(X)
}

void main() {
   var X = 3
   call f1()
}
```

Assume that the programming language implements variable hiding and that execution starts with the `main` subprogram. **Provide** only the output of the above program code assuming the following scoping rules are used by the programming language. Provide only the numeric output (do not include additional output such as spaces or quotes) if output is successfully generated, and the text "Error" (without the quotes) if an error or invalid output is produced.

Static scoping: _____

Dynamic scoping: _____

2 points

QUESTION 11

1. Some programming languages support complex numbers as a primitive data type. Consider the following statements, and **select** the one that most accurately describes an implication of such support.

- Computations using complex number types have a somewhat high cost of execution, because these computations are not directly supported by most hardware.
- Computations using complex number types have a very low cost of execution, because these computations are directly supported by most hardware.
- Support for complex number types greatly increases the readability of a programming language.
- Support for complex number types greatly decreases the writability of a programming language.

1 points

QUESTION 12

1. Consider the following statements, and **select** the one that most accurately describes a potential drawback associated with Boolean data types.

- Operations on Boolean data types have a high execution cost.
- Boolean data types are inefficient in terms of memory space.
- Boolean data types have low readability in most modern programming languages.
- Boolean data types are never considered to be ordinal.

1 points

QUESTION 13

1. Consider the following enumeration type definition, in a hypothetical programming language:

```
enumeration month {jan, feb, mar}
```

Furthermore, assume that in this language, other types can be coerced into enumeration types. Consider the following snippets of program code, and **select** the one that correctly represents an error that could occur due to this coercion..

- ```
  month firstMonth = feb
  month secondMonth = mar
  print(firstMonth + secondMonth)
  ```
- ```
  month myMonth = 1
  ```
- ```
  month myMonth = 4
  ```
- ```
  print(jan)
  ```

1 points

QUESTION 14

1. Consider the following statements about index range checking for arrays in Ada and Java, and **select** the one that is the most correct.

- Ada's index range checking is always less reliable than Java's range checking.
- Ada's index range checking can be more reliable than Java's range checking.
- Ada's index range checking is much less writable than Java's range checking.
- Ada's index range checking is much more writable than Java's range checking.

1 points

QUESTION 15

1. Consider the following program code in a hypothetical programming language:

```
void f() {
   integer myVar = 3
   integer myArray[myVar]

   myArray[0] = 5
   myArray[1] = 2
   myArray[2] = 4

   print(myArray[1])
}
```

Consider the following statements, and **select** the one that most correctly describes the category of `myArray`.

- ○ Static
- ○ Fixed stack-dynamic
- ○ Stack-dynamic
- ○ Fixed heap-dynamic
- ○ Heap-dynamic

1 points

QUESTION 16

1. Assume that you wish to simulate the functionality of a hetergeneous array in a programming language that does not directly support heterogeneous arrays. Consider the following options, and **select** only the approaches that will correctly simulate the functionality of a heterogeneous array in the programming language. Note that incorrectly selected options will be penalised.

- ☐ Using a normal array in a statically typed programming language, which stores values of different primitive data types.
- ☐ Using a normal array in a statically typed programming language, which stores instances of an `Object` class, where all other classes inherit from the `Object` class.
- ☐ Using two normal arrays in a statically typed programming language, where one stores keys and the other stores corresponding values.
- ☐ Using a normal array in a dynamically typed programming language.

2 points

QUESTION 17

1. Matrices in C do not support array slices. Consider the following options, and **select** the one that most correctly describes the reason for this.

- ○ C implements matrices using arrays containing other arrays. Arrays in C have built-in slice operations, meaning the matrix itself does not require support for slice operations.
- ○ C implements matrices using arrays containing other arrays. It is therefore easy to retrieve part of the matrix using standard array operations.
- ○ C implements matrices using rectangular arrays. Rectangular arrays cannot support slice operations.
- ○ C implements matrices using rectangular arrays. Rectangular arrays cannot represent matrices.

1 points

QUESTION 18

1. Consider the following statements about the `MOVE CORRESPONDING` operation in COBOL, and **select** the one that is the most correct.

- The `MOVE CORRESPONDING` operation in COBOL is equivalent to an assignment between records in most modern imperative programming languages.
- The `MOVE CORRESPONDING` operation in COBOL is useful because it concatenates two records, which is a common operation that often needs to be performed in the programming domain for which COBOL is intended.
- The `MOVE CORRESPONDING` operation in COBOL is useful because it extracts sub-records within another record, which is a common operation that often needs to be performed in the programming domain for which COBOL is intended.
- The `MOVE CORRESPONDING` operation in COBOL is useful because it copies only relevant data between two records, which is a common operation that often needs to be performed in the programming domain for which COBOL is intended.

1 points

QUESTION 19

1. Consider the following statements about list comprehensions, and **select** the one that is the most correct.

- Support for list comprehensions greatly increases the writability of lists and list operations.
- Support for list comprehensions greatly increases the readability of lists and list operations.
- Support for list comprehensions greatly increases the orthogonality of lists and list operations.
- Support for list comprehensions greatly decreases the amount of memory space occupied by lists.

1 points

QUESTION 20

1. Some programming languages languages implicitly dereference all pointers. Consider the following statements about such languages, and **select** the one that is the most correct.

- Implicitly dereferenced pointers are more reliable than explicitly dereferenced pointers.
- Implicitly dereferenced pointers prevents pointers from being used for indirect addressing.
- Implicitly dereferenced pointers prevents pointers from being used for dynamic storage management.
- Implicitly dereferenced pointers disallow pointer arithmetic.

1 points